Case Study: Parallel Reduction

Thursday, April 2, 2026 4:16 PM

AI: $\frac{N-1}{4N+4} \simeq \frac{1}{4} \Rightarrow$ BW bound

GB

kB ⇒ 1024 B

kB/s ⇒ 1000 B/s

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;
    if (index < blockDim.x) {
        sdata[index] += sdata[index + s];
    }
    __syncthreads();
}
```

W0 of TB0. tid: 0~31

s = 1

index = 2 · 1 · tid. = 0, 2, 4, ... 62.

sdata[0, 2, ...] += sdata[0, 2 ... 62 + 1]

tid=0 → sdata[0] = sdata[0] + sdata[1]

tid=1 → sdata[2] = sdata[2] + sdata[3]

⋮

s = 2, ...

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;
    if (index < blockDim.x) {
        sdata[index] += sdata[index + s];
    }
    __syncthreads();
}
```

→

Shared Memory Accesses

reads: sdata[index], sdata[index + s]

write: sdata[index].

TB(0), warp 0. → tid → 0~31

s = 1. index = 2 · tid → 0, 2, 4, ... 62

sdata[index] ⇒ sdata[0, 2, 4, ..., 62] → 2-way Bank conflicts

sdata[index + s] ⇒ sdata[1, 3, 5, ... 63] → 2-way bank conflicts

s = 2 index = 4 · tid → 0, 4, 8, ..., 124

sdata[index] → 4-way bank conflict

s = 4, 8, 16, ...

Trade off on # of TBs vs. # of work for each thread.

N → N/n threads

TB size: 1k

ver1. N = 10k, n = 10, ⇒ 1k threads. 1 TB

ver2. N = 10k, 5k threads ⇒ 5 TBs.

On a GPU with 5 SMs.

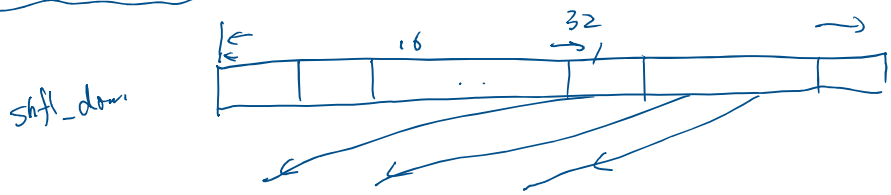
```
__inline__ __device__ float warpReduceSum(float val) {
    for (int offset = 16; offset > 0; offset /= 2) {
        val += __shfl_down_sync(0xFFFFFFFF, val, offset);
    }
    return val;
}
```

⇐

```
__device__ void warpReduce(volatile int* sdata, int tid) {
    if (blockSize >= 32) sdata[tid] += sdata[tid + 16];
    if (blockSize >= 16) sdata[tid] += sdata[tid + 8];
    if (blockSize >= 8) sdata[tid] += sdata[tid + 4];
    if (blockSize >= 4) sdata[tid] += sdata[tid + 2];
    if (blockSize >= 2) sdata[tid] += sdata[tid + 1];
}
```

offset = 16

val += __shfl_down_sync(0xFFFF, val, 16)



val = val + __shfl_

